

Discrete-Event Simulation and Constrained Optimisation of Emergency-Department Patient Flow*

A From-Scratch Replication of Lv et al. (2026) and Four Extensions

Sudip Kumar Saha

Student ID 2105152

CSE, BUET

2105152@ugrad.cse.buet.ac.bd

Ahnaf Akib Tanim

Student ID 2105154

CSE, BUET

2105154@ugrad.cse.buet.ac.bd

Md. Shahjalal Rumman

Student ID 2105165

CSE, BUET

2105165@ugrad.cse.buet.ac.bd

Md. Dulal Hossain

Student ID 2105169

CSE, BUET

2105169@ugrad.cse.buet.ac.bd

Md. Shoaib Hossain

Student ID 2105170

CSE, BUET

2105170@ugrad.cse.buet.ac.bd

Abstract

Every time a physician in an emergency department (ED) becomes free, someone has to decide which of three queues to serve next: newly arrived urgent patients, newly arrived non-urgent patients, or patients coming back for a follow-up after their diagnostics. Lv et al. (2026) studied this problem with a discrete-event simulation (DES) of a Chinese tertiary-hospital ED and compared three physician-scheduling heuristics. We rebuilt that simulation from scratch, checked it against every KPI the paper publishes, and then extended it in four ways. First, we added the $c\mu$ rule and its generalisation, two scheduling policies that come with optimality results for a stated delay-cost objective instead of being designed by hand. Second, we replaced the paper's two-stage grid search over the slack thresholds (k_1, k_2) with our own Nelder–Mead implementation, using a sample-average approximation and an exterior penalty for the service-level constraint. Third, we built a weekly cost model and used it to optimise the integer physician roster together with the thresholds. Fourth, we used exact common random numbers (CRN) and measured how much variance they remove. Our rebuild reproduces all six structural findings of the paper, and its equipment-utilisation figures agree with the published table to within 0.5 percentage points. Absolute waiting times come out 12–25% higher, which we attribute to the high implied load $\rho \approx 0.87$. Nelder–Mead gets within 0.006 minutes of the grid-search optimum while using $19\times$ fewer simulations, the cost-optimal roster lowers weekly cost by 20.7%, and CRN removes 98.9% of the variance in policy comparisons. We also found that the thresholds published in the paper do not satisfy the service-level constraint on our model, so they need to be re-tuned rather than reused.

CCS Concepts

• **Applied computing** → **Health informatics**; • **Computing methodologies** → **Discrete-event simulation**; • **Theory of computation** → **Scheduling algorithms**; • **Mathematics of computing** → *Nonconvex optimization*.

Keywords

discrete-event simulation, emergency department, patient scheduling, $c\mu$ rule, Nelder–Mead, simulation optimisation, common random numbers, variance reduction, staffing

1 Introduction and Base Paper Review

1.1 Background

Emergency departments are one of the main congestion points in a hospital. Demand is random and changes a lot over the day and the week, service times vary from patient to patient, and the key resource, physician time, is limited and expensive. Patients are triaged into urgency levels that each have their own waiting target, so an ED can be modelled as a multi-class queue with deadlines. Simulation is the usual way to evaluate operating policies in this setting [16, 18], since time-varying arrivals, non-exponential service and history-dependent priority rules make exact analysis impractical [5].

1.2 The base paper

Our base paper is Lv, Liu, Yan and Wang [10]. It builds a DES of a Grade-A tertiary-hospital ED in China and limits it to triage Level III (urgent) and Level IV (non-urgent) patients, who make up most ED visits. Patients arrive according to a non-homogeneous Poisson process whose rate $\lambda_{d,h}$ depends on the day and hour. Each patient waits for an *initial* consultation and may then need diagnostics (laboratory, X-ray, CT or B-ultrasound). Patients who do need diagnostics come back afterwards and compete for the same physicians in a *follow-up* queue. Physicians work three shifts per day, staffed (5, 5, 3).

The paper compares three physician-scheduling policies within one simulation framework:

- **Initial-First (IFP)**: new arrivals always come first, Level III before Level IV, and follow-ups are served only when there is capacity left over.
- **Alternating 1:1 (ALT)**: at most half the physicians may do initial consultations at any time, and the rest take follow-ups.
- **Slack-Based (SBP)**: a patient of level ℓ is flagged *urgent* once their waiting time reaches $T_\ell - k_\ell$, where T_ℓ is the

*CSE 402: Numerical Analysis, Simulation and Modeling Sessional. **Group 02, Section C**. Code and all generated results: <https://github.com/Ahnaf-Akib-Tanim/Numeric-Project-submission>

Table 1: Gaps in the base paper and the corresponding contribution of this work.

Paper does	Paper does not	Our work
Grid-search two thresholds	Optimise the <i>staffing level</i> jointly	\$2.7
Compare three heuristics	Compare against a provably optimal rule	\$2.5
Brute-force search	Use a derivative-free numerical optimiser	\$2.6
100 independent replications	Apply variance reduction	\$2.8
Report results	Release engine or data	\$3

triage target and k_t is a tunable slack parameter [3]. The priority order is then urgent-L3 > urgent-L4 > follow-up > non-urgent initial.

The SBP thresholds are tuned by a two-stage grid search (coarse step 1.0, then fine step 0.1). The paper reports an optimum of $(k_1, k_2) = (13.1, 2.1)$ and a drop in mean waiting time from 46.26 min under IFP to 35.25 min, a 23.8% reduction, with no service-level violations. Significance is tested with paired t -tests, a Bonferroni correction and Cohen’s d .

1.3 Gaps and objectives

The base paper is a careful comparison, but there are several things it does not do, listed in Table 1. Its source code and operational dataset are also not public. Only summary statistics are published, but these turned out to be enough to rebuild an equivalent simulator.

Our objectives were therefore to: (i) rebuild the model from scratch and see how closely the published results can be reproduced from the published parameters alone; (ii) add a scheduling rule that comes from queueing theory instead of being designed by hand; (iii) replace the grid search with a proper numerical optimiser under an explicit constraint, and measure the saving in a way that does not depend on hardware; (iv) answer the staffing question that the paper only looks at through sensitivity analysis; and (v) make the statistical comparison sharper through variance reduction.

2 Methodology

2.1 System model

Let t be simulated time in minutes over a horizon of one week ($T = 10,080$). As in the paper, the day and hour indices are

$$d = \lfloor \frac{t}{1440} \rfloor \bmod 7, \quad h = \lfloor \frac{t \bmod 1440}{60} \rfloor, \quad (1)$$

so the arrival rate $\lambda_{d,h}$ is piecewise constant and repeats every week. Each arrival gets triage level $\ell \in \{3, 4\}$ with probabilities 0.25 and 0.75, and the matching waiting target $T_3 = 30$ or $T_4 = 120$ minutes. The state consists of three FIFO queues, $W_{\text{init},3}^t$, $W_{\text{init},4}^t$ and W_{follow}^t , which share R^t physicians.

2.2 Discrete-event engine

The simulator is our own event loop built on a binary min-heap. We did not use a process-oriented simulation library, so every mechanism the paper describes appears explicitly in the code. Each event

Algorithm 1 Discrete-event main loop

```

1: push all arrivals and shift changes onto the heap  $H$ 
2: while  $H \neq \emptyset$  do
3:    $e \leftarrow \text{POP}_{\text{MIN}}(H)$ 
4:   if  $e.\text{time} \geq T$  then break
5:   end if
6:    $\text{now} \leftarrow e.\text{time}$ ; accrue utilisation integrals
7:   apply the state transition of  $e$ 
8:    $R \leftarrow \text{FREE}_{\text{PHYSICIANS}}(\text{now})$ 
9:   if  $R > 0$  and some queue is non-empty then
10:     $P \leftarrow \text{POLICY.ALLOCATE}(\text{state}, \text{now}, R)$ 
11:    start service for every patient in  $P$ 
12:   end if
13: end while

```

is a 4-tuple $(\text{time}, \text{type}, \text{seq}, \text{payload})$ ordered lexicographically. The increasing counter seq breaks ties, which makes the ordering total and each replication exactly reproducible. There are five event types: shift change, arrival, consultation end, examination device release and examination results ready. After each event the engine calls a single DISPATCH routine, and this is the only place where the scheduling policy is applied (Algorithm 1). Changing the policy therefore touches no other part of the simulator, so all policies run under the same mechanics.

Service is non-preemptive. When staffing drops at 22:00, consultations already in progress continue, and no new consultation starts until the number of busy physicians falls below the new capacity.

2.3 Arrival generation by thinning

Since $\lambda(t)$ is a step function with 168 pieces, inverting its cumulative intensity is easy to get wrong. We sample the process by Lewis–Shedler thinning [9] instead. With $\lambda^* = \max_{d,h} \lambda_{d,h} = 23.05/\text{h}$, we generate a homogeneous process of rate λ^* by adding up $\text{Exp}(1/\lambda^*)$ gaps, and keep each candidate time t with probability $\lambda(t)/\lambda^*$. In a small interval dt a candidate appears with probability $\lambda^* dt$ and is kept with probability $\lambda(t)/\lambda^*$, so the kept points have intensity $\lambda(t)$ exactly. About 58% of candidates are accepted.

2.4 Service times by inverse-CDF sampling

Consultation times follow truncated exponentials $\text{TruncExp}(\lambda_k, [a_k, b_k])$. Inverting the truncated CDF gives

$$x = -\frac{1}{\lambda} \ln \left(e^{-\lambda a} - u (e^{-\lambda a} - e^{-\lambda b}) \right), \quad u \sim U(0, 1), \quad (2)$$

which uses exactly one uniform per variate. We chose this on purpose. Rejection sampling uses a variable number of uniforms, which would break the stream synchronisation that §2.8 relies on. The mean after truncation has a closed form,

$$\mathbb{E}[X] = a + \frac{1}{\lambda} - \frac{(b-a) e^{-\lambda(b-a)}}{1 - e^{-\lambda(b-a)}}, \quad (3)$$

which gives 9.093 min for initial and 12.841 min for follow-up consultations. One detail of the paper needs comment here. A truncated exponential on $[5, 25]$ cannot have a mean of 15 min, because 15 is only the limit as $\lambda \rightarrow 0$ and is never reached. So $\lambda_2 = 1/15$ is the only reading that fits the paper’s stated bounds, and it is also the

Table 2: Index values computed by the $c\mu$ and generalised $c\mu$ rules under the base paper's parameters.

Class	T_ℓ	$\mathbb{E}[S_\ell]$	c_ℓ	$c_\ell \mu_\ell$
Level III initial	30	9.093	0.03333	0.003666
Follow-up	60	12.841	0.01667	0.001298
Level IV initial	120	9.093	0.00833	0.000916

reading that reproduces the reported physician utilisation of about 85%.

2.5 Scheduling policies

We implement IFP, ALT and SBP directly from Equations (8)–(23) of the base paper, and add two policies taken from queueing theory.

The $c\mu$ rule (WSEPT). Cox and Smith [2], with a later proof by Buyukkoc et al. [1], showed that for a multi-class queue served by a single pool with *linear* holding costs c_ℓ per unit of waiting, expected total holding cost is minimised by always serving the non-empty class with the largest index

$$I_\ell = c_\ell \mu_\ell, \quad \mu_\ell = 1/\mathbb{E}[S_\ell]. \quad (4)$$

We set $c_\ell = 1/T_\ell$, so a minute of waiting costs more for classes that can tolerate less waiting. This turns the triage targets directly into delay costs. Table 2 lists the resulting indices. The rule is also known as Weighted Shortest Expected Processing Time (WSEPT). Because its indices are constants, it reduces to a fixed priority order (L3 initial > follow-up > L4 initial). This follows from the theorem itself, and it is the limitation that the generalised rule addresses.

The generalised $c\mu$ rule (Gc μ). For *convex* delay costs $C_\ell(w)$, Van Mieghem [15] and Mandelbaum and Stolyar [11] show that the asymptotically optimal index is $C'_\ell(w) \mu_\ell$, evaluated at the head-of-line delay. With $C_\ell(w) = (w/T_\ell)^2$ this becomes

$$I_\ell(t) = \frac{2 w_{\text{head}} \mu_\ell}{T_\ell^2}, \quad (5)$$

so the index increases with how long the patient at the front of the queue has been waiting. Like SBP, Gc μ reacts to waiting time, but it follows from the cost function rather than from tuning, and it has no free parameters.

A fair objective. The $c\mu$ rule is optimal for holding cost, not for the unweighted mean wait, so it would be unfair to judge it on the mean wait alone. We therefore also report the *holding cost*

$$H = \frac{1}{N} \sum_{i=1}^N \frac{w_i}{T_{\ell(i)}}, \quad (6)$$

which measures each patient's wait relative to their own clinical target.

2.6 Constrained simulation optimisation

The tuning problem in the base paper is

$$\min_{k_1, k_2} \bar{W}_{\text{total}}(k_1, k_2) \quad \text{s.t.} \quad SL_\ell(k_1, k_2) \geq SL_\ell^{\min}, \quad \ell \in \{3, 4\}. \quad (7)$$

We needed three changes to make it suitable for a simplex method.

Algorithm 2 Nelder–Mead iteration (minimisation)

```

1: order vertices so that  $f(x_1) \leq \dots \leq f(x_{n+1})$ 
2: stop if simplex span  $\leq x_{\text{tol}}$  and  $f(x_{n+1}) - f(x_1) \leq f_{\text{tol}}$ 
3:  $x_c \leftarrow \frac{1}{n} \sum_{i \leq n} x_i$  ▷ centroid without worst
4:  $x_r \leftarrow \Pi(x_c + \alpha(x_c - x_{n+1}))$  ▷ reflect
5: if  $f(x_r) < f(x_1)$  then
6:    $x_e \leftarrow \Pi(x_c + \gamma(x_r - x_c))$  ▷ expand
7:    $x_{n+1} \leftarrow \arg \min\{f(x_e), f(x_r)\}$ 
8: else if  $f(x_r) < f(x_n)$  then
9:    $x_{n+1} \leftarrow x_r$ 
10: else
11:    $x_k \leftarrow \Pi(x_c + \rho(\cdot - x_c))$  ▷ contract
12:   if improved then  $x_{n+1} \leftarrow x_k$ 
13:   else  $x_i \leftarrow \Pi(x_1 + \sigma(x_i - x_1)) \quad \forall i > 1$ 
14:   end if
15: end if

```

Sample average approximation. Simulation outputs are noisy, and simplex methods tend to stall on noisy surfaces. We therefore minimise the average over a *fixed* set of n patient streams [7]. Since the engine is deterministic for a given stream, $\hat{F}$ is a deterministic function, and evaluating the same point twice gives the same value. Using the same streams for every candidate also cancels much of the noise between points, so the surface is far smoother than the noise of a single run would suggest.

Exterior penalty. Nelder–Mead only handles unconstrained problems, so we add the service-level constraint as a penalty:

$$F(k_1, k_2) = \bar{W}_{\text{total}} + \mu \sum_{\ell \in \{3, 4\}} \max(0, SL_\ell^{\min} - SL_\ell), \quad (8)$$

with $\mu = 500$ minutes per unit of shortfall. In other words, missing the service level by one percentage point costs the same as five extra minutes of average waiting [13]. This is large enough that an infeasible point never comes out as the optimum, but small enough that the penalised surface stays smooth near the boundary. The bounds $k \geq 0$ are enforced by *projection*, which lets the simplex move along a bound instead of being pushed away from it.

Nelder–Mead. We wrote the downhill simplex method [12] ourselves (Algorithm 2) with the standard coefficients $\alpha = 1$, $\gamma = 2$, $\rho = 0.5$, $\sigma = 0.5$. Two details mattered in practice. First, the initial simplex uses a separate step for each dimension, (4, 15), because k_1 ranges over roughly $[0, 30]$ (against $T_3 = 30$) while k_2 ranges over roughly $[0, 120]$ (against $T_4 = 120$). A single step size would give a poorly conditioned simplex. Second, since the method is local, we run it from four different starting points and report how much the results differ.

Here $\Pi(\cdot)$ is projection onto the box. We measure the optimiser's cost in *week-long simulations* rather than seconds, so the comparison with the paper's grid does not depend on hardware or on how many processes run in parallel.

2.7 Cost-based staffing

The paper varies staffing levels but cannot recommend one, because it has no way to compare the cost of a physician-hour with the cost

of a minute of patient waiting. We add that missing objective:

$$C(R, k_1, k_2) = \alpha H_{\text{phys}} + \beta W_{\text{total}} + \gamma V_{\text{SLA}}, \quad (9)$$

where $R = (R_m, R_d, R_n)$ is the integer roster, H_{phys} is rostered physician-hours per week, W_{total} is total patient waiting-minutes per week and V_{SLA} is the number of patients who waited past their triage target. Good thresholds for a night shift with 3 physicians are not the same as good thresholds for one with 5, so we re-optimize (k_1, k_2) inside each candidate roster. Roster and thresholds are therefore optimised together rather than one after the other. The search has two stages. First, we screen every roster in $\{2, \dots, 8\}^3$, except those with more physicians at night than during the day, and drop the unstable ones. A roster counts as unstable if more than 15% of the week’s patients are still in the system at the end of the horizon, which means the queue is growing instead of clearing. The best remaining candidates then get a full Nelder–Mead re-tune.

2.8 Common random numbers and inference

For two policies A and B ,

$$\text{Var}(A - B) = \text{Var}(A) + \text{Var}(B) - 2 \text{Cov}(A, B). \quad (10)$$

With independent streams the covariance is zero. Under CRN both policies see the same simulated week, so a busy week raises both results together. The covariance becomes strongly positive and the variance of the *difference* drops sharply, while each policy’s own variance stays the same [8].

In practice much of this benefit is often lost through desynchronisation: once two policies start making different decisions, they use random numbers in a different order. We prevent this by design. From one seed we spawn six independent PCG64 substreams [14] (arrivals, triage levels, initial service, follow-up service, examination need, examination selection). The whole exogenous patient stream, including arrival times, levels, both consultation durations and required diagnostics, is generated *before* the run starts, so the engine draws no random numbers during the simulation. Each variate also uses exactly one uniform, by Eq. (2). As a result, patient #1723 (or any other) is identical under every policy.

For inference we follow the base paper: paired t -tests on the per-replication means (paired because CRN makes the design paired), a Bonferroni correction across the family of comparisons, and Cohen’s d in its paired form $\bar{d}/s_d$. We also report the smallest n that satisfies $t_{0.975, n-1} s/\sqrt{n} \leq h$, found by iteration because the t quantile itself depends on n .

3 Implementation and Experiments

3.1 Software

The simulator is written in Python 3.11 and has about 7,900 lines in total: a 15-module library, 10 experiment scripts and a verification suite with 33 checks. We use only NumPy [4], SciPy [17], Matplotlib [6] and pandas. SciPy is used only for distribution quantiles, the t -distribution, and as an independent check on our own optimiser. The library follows the five-module structure of the base paper’s Figure 2 (entity definition, event scheduling, resource scheduling, strategy execution and statistical output), so the code and the paper can be read side by side. All code is available at <https://github.com/Ahnaf-Akib-Tanim/Numeric-Project-submission>.

Table 3: Model parameters. All values are from the base paper except the five marked [†], which it does not specify.

Parameter	Value
Horizon / replications	10,080 min (1 week) / 100
Arrival rate $\lambda_{d,h}$	Table 2, 7×24 ; peak 23.05/h
Triage mix	25% Level III, 75% Level IV
Targets T_3, T_4	30 min, 120 min
Initial consultation	TruncExp(1/9) on [5, 15]
Follow-up consultation	TruncExp(1/15) on [5, 25]
$P(\text{diagnostics})$	0.60
Modality probabilities	Lab .92, CT .55, X-ray .29, US .22
Modality durations	1.19, 2.45, 3.99, 6.58 min
Report delays	20, 30, 30, 0 min
Shifts (M/A/N)	5 / 5 / 3 physicians
$SL_{\ell}^{\min}$	0.95 [†]
$T_{\text{follow}}(c\mu)$	60 min [†]
Search box	$k_1 \in [0, 30], k_2 \in [0, 60]$ [†]
Cost weights α, β, γ	120, 0.50, 50 [†]
“At least one” exam rule	resample [†]
Base seed	20250402

One implementation detail made a large difference. We first stored events as an ordered dataclass, and profiling showed that Python-level comparisons inside the heap took 22% of the runtime. Switching to plain tuples moved these comparisons into C and reduced the time per replication from 0.20 s to 0.076 s. Without this change, the 50,415-simulation grid search in §4.3 would not have been practical.

3.2 Parameters

All parameters come from the base paper and are listed in Table 3. The arrival rates are copied directly from its Table 2 (7×24 values). The offered load implied by these parameters is

$$\rho = \frac{N(\mathbb{E}[S_1] + p_{\text{exam}}\mathbb{E}[S_2])}{\sum_s L_s R_s \cdot 7} = 0.870, \quad (11)$$

which agrees with the 84.98–85.43% physician utilisation reported in the paper. This gave us a check on the parameters before running any simulation.

3.3 Parameters we had to choose

The paper leaves five quantities unspecified. $SL_{\ell}^{\min}$ appears as a constraint but is never given a value. We use 0.95, a common ED target, which is strict enough for the penalty to matter. T_{follow} , which the $c\mu$ rule needs, is set to 60 min, halfway between T_3 and T_4 ; varying it from 15 to 120 min does not change the policy ranking. The search box has to include at least the paper’s reported winners at $k_1 = 26$ and $k_2 = 40$. The cost weights only exist in our extension and are tested in §4.4. Lastly, the paper says that 60% of patients need “at least one” test, but its independent per-modality probabilities select no test at all with probability 1.99%. We resample in that case so that every patient sent for diagnostics gets at least one test.

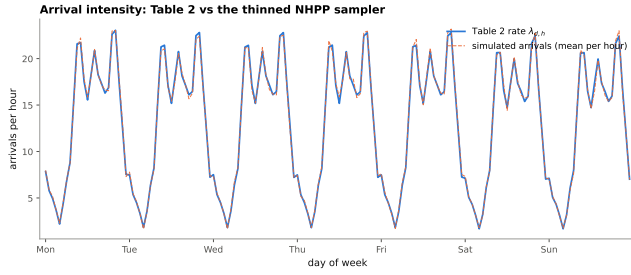


Figure 1: Validation of the thinned non-homogeneous Poisson arrival process against the base paper’s published hourly rates.

3.4 Experimental design

Each comparison uses 100 replications of one simulated week per policy under CRN, the same number of replications as the base paper. Replication r uses the same patient stream for every policy, so all comparisons are paired. We ran the experiments on a 16-core machine with 14 worker processes. The full pipeline takes about 20 minutes, and about half of that is the grid search.

3.5 Verification

We check correctness with 33 automated assertions, each tied to a property that some reported number depends on. They cover the samplers against their closed forms (Eq. (3) against 200,000 draws), the day/hour indexing, convergence of the thinned arrival process, CRN seed behaviour, and engine invariants: no patient is served before arriving, no physician sits idle while a patient waits (work conservation, checked for every policy), examinations happen in sequence, and shift changes are non-preemptive. The optimiser is tested on the Rosenbrock function and against SciPy. All 33 checks pass.

4 Results and Discussion

4.1 Model validation

Figure 1 checks the input process. Over 200 sampled weeks, the simulated hourly arrival counts match the paper’s Table 2 with a relative L_1 error of 1.67% and a correlation of 0.9991.

Table 4 compares every KPI the paper publishes. Our model matches the paper’s patterns more closely than its exact numbers. The results that agree closely are both service levels under IFP (100%/100%), ALT’s Level III waiting time (6.05 vs. 6.82 published), SBP’s Level III waiting time (12.35 vs. 10.48), physician utilisation (86.3% vs. 85.0–85.4%) and the fact that it barely changes across policies, and the equipment-utilisation table (Table 5), where all four modalities are within 0.5 percentage points. This supports the paper’s main managerial conclusion that physicians, not equipment, are the bottleneck.

The absolute waiting times do not match: ours are 12–25% higher. We think this comes down to load. The published parameters imply $\rho \approx 0.870$, and close to saturation the mean wait is very sensitive to small changes in utilisation. A small difference in the patient mix, which we cannot recover, can therefore cause a large difference in waiting time. Since the original dataset is not public, an exact

Table 4: Validation against the base paper. Deviation is relative to the published value; 100 replications.

Policy	Metric	Ours	Paper	Dev.
IFP	Mean wait, all (min)	52.43	46.26	+13.3%
	Mean wait, Level III	50.88	44.14	+15.3%
	Service level L3 / L4	100 / 100	100 / 100	0.0%
	Physician utilisation	86.36	85.43	+1.1%
ALT	Mean wait, all (min)	42.39	37.88	+11.9%
	Mean wait, Level III	6.05	6.82	−11.2%
	Service level L3 / L4	99.96 / 87.37	98.08 / 89.29	−2.2%
	Physician utilisation	86.34	85.41	+1.1%
SBP	Mean wait, all (min)	44.08	35.25	+25.1%
	Mean wait, Level III	12.35	10.48	+17.9%
	Service level L3 / L4	99.99 / 89.40	100 / 100	−10.6%
	Physician utilisation	86.34	84.98	+1.6%

Table 5: Diagnostic equipment utilisation, reproduced from the base paper’s Table 9.

Equipment	Exams/week	Ours	Paper
Laboratory	1248	14.73%	14.36%
X-ray	394	15.59%	15.20%
CT	748	18.18%	17.64%
B-ultrasound	297	19.41%	19.03%

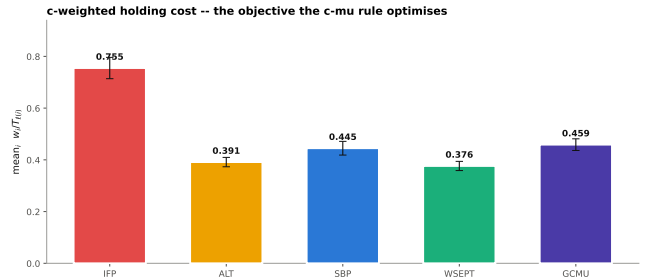


Figure 2: Holding cost by policy, the objective the $c\mu$ rule is optimal for. Each patient’s wait is measured relative to their own clinical target (Eq. (6)). Lower is better.

numerical match is not possible and we do not claim one. We report the gap as it is rather than adjusting parameters to hit the published numbers.

4.2 Policy comparison

Table 6 brings together all six policies, and Figure 3 shows the same results graphically. WSEPT has the lowest holding cost (0.3763 compared with 0.7550 for IFP, a 50% reduction), as the Cox–Smith result predicts. On this system it also has the lowest unweighted mean wait (42.32 min). All of its differences from the paper’s three heuristics remain significant after Bonferroni correction (Table 7).

The service-level columns of Table 6 show the main trade-off: the policies that serve Level III patients fastest are the same ones that let Level IV patients miss their target. WSEPT reaches only 86.9%

Table 6: Comparison of all six scheduling policies; 100 common-random-number replications. SBP* is the Slack-Based Policy with our constrained Nelder–Mead thresholds. “Meets SLA” requires $SL_\ell \geq 95\%$ on *both* triage levels.

Policy	Origin	Wait (min)	95% CI	L-III	L-IV	Holding	SL_3	SL_4	Util.	Meets SLA
IFP	base paper	52.43	[49.63, 55.22]	50.88	52.94	0.7550	100.00%	100.00%	86.36%	yes
ALT	base paper	42.39	[40.26, 44.51]	6.05	54.61	0.3913	99.96%	87.37%	86.34%	no
SBP	base paper (13.1, 2.1)	44.08	[41.72, 46.44]	12.35	54.74	0.4450	99.99%	89.40%	86.34%	no
SBP*	ours (2.31, 8.37)	44.18	[41.81, 46.55]	14.14	54.27	0.4570	95.42%	95.64%	86.34%	yes
WSEPT	ours , $c\mu$	42.32	[40.21, 44.43]	3.76	55.30	0.3763	100.00%	86.87%	86.34%	no
$Gc\mu$	ours , gen. $c\mu$	44.05	[41.81, 46.29]	14.59	53.94	0.4588	100.00%	90.99%	86.35%	no

Table 7: Paired t -tests on the holding cost, WSEPT against every other policy. Bonferroni-adjusted $\alpha = 0.00333$.

Comparison	Diff.	t	p	Cohen’s d
IFP vs. WSEPT	+0.3786	31.67	1.3×10^{-53}	3.17
ALT vs. WSEPT	+0.0150	36.54	2.9×10^{-59}	3.65
SBP vs. WSEPT	+0.0686	14.15	1.6×10^{-25}	1.42
WSEPT vs. $Gc\mu$	-0.0824	-33.42	1.0×10^{-55}	-3.34

Table 8: Simulation budget required by brute-force grid search and by our Nelder–Mead simplex, on the same objective and the same patient streams.

Method	Points	Simulations	Objective
Grid, coarse (step 1.0)	1,891	28,365	47.6105
Grid, fine (step 0.1)	441	22,050	46.4601
Grid, total	2,332	50,415	46.4601
Nelder–Mead, 1 start	53	2,650	46.4668
Nelder–Mead, 4 starts	196	9,800	46.4668

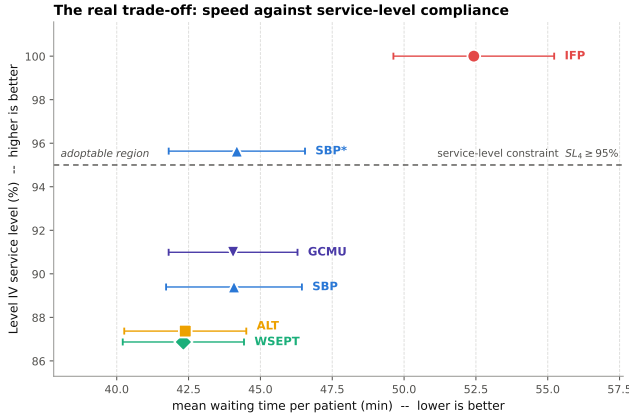


Figure 3: Speed against service-level compliance. Only policies above the dashed constraint line can be adopted. Among the tuned policies, only SBP* is both fast and feasible.

on Level IV, ALT is similar, and the paper’s own tuned SBP reaches 89.4%. Only IFP, which is slow, and our re-tuned SBP* meet the constraint on both levels. So there is no single best policy, and the answer depends on the objective. WSEPT gives the lowest holding cost and the lowest raw mean wait, while SBP* is the fastest policy a hospital could adopt under the 95% constraint.

There is one caveat. The Cox–Smith optimality proof assumes a single-stage multi-class queue. Our model has re-entrant flow, since patients come back for a follow-up after diagnostics, and the number of physicians changes over time. The optimality guarantee therefore holds only approximately here, so we rely on the observed ranking rather than on the theorem.

4.3 Optimiser versus brute force

Table 8 shows the main efficiency result. When we ran the paper’s procedure on our model, the two-stage grid evaluated 2,332 points. That took 50,415 week-long simulations, roughly 970 simulated patient-years of ED operation, and ended at (4.10, 8.20). Our simplex found an equally good point with 2,650 simulations, which is 19.0 times fewer. When both optima were re-evaluated on the same full-length batch, they differed by 0.006 minutes, well below the noise of the simulation.

Starting from four different points, the runs ended with objective values within 0.15 min of each other (46.468–46.622). This suggests that the penalised surface has a single minimum inside this box. Our implementation also agrees with `scipy.optimize` to 0.0000 min on the same objective. Figure 4 shows the simplex path over the response surface that the grid had to evaluate point by point, and Figure 5 plots solution quality against budget.

This experiment also gave a result we did not expect. With the 95% constraint in place, the optimum shifts to a much larger k_2 , and the paper’s published (13.1, 2.1) becomes *infeasible* on our model, reaching only 89.4% on Level IV (Table 9). The reason is simple. With $T_4 = 120$ and $k_2 = 2.1$, a Level IV patient is flagged urgent only after waiting 117.9 of their 120 allowed minutes, which leaves too little time to see them in a system this congested. This is the clearest case in our work for re-tuning under an explicit constraint instead of reusing published parameters.

4.4 Cost-optimal staffing

Table 10 prices the paper’s seven staffing scenarios using Eq. (9). On its own, the paper can only say that more physicians give shorter waits, which does not tell a hospital where to stop. Once the scenarios are priced, the cost is lowest at S_5 . A free search over all 140 admissible rosters, with (k_1, k_2) re-tuned inside each one,

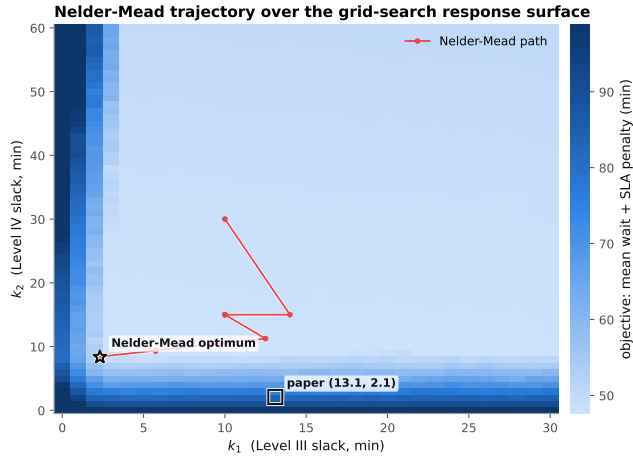


Figure 4: The Nelder–Mead path over the grid-search response surface. The dark band is the service-level penalty region; the base paper’s published optimum (square) lies inside it.

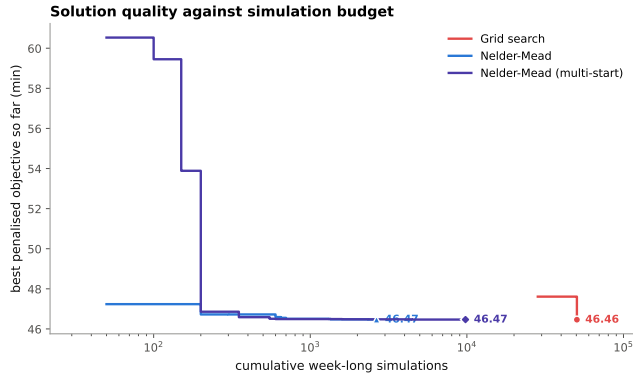


Figure 5: Best objective found so far against cumulative simulation budget (log scale).

Table 9: All three optima re-evaluated on the same replications.

Setting	(k_1, k_2)	Wait	SL_3	SL_4	Status
Nelder–Mead	(2.31, 8.37)	44.18	95.42%	95.64%	feasible
Grid search	(4.10, 8.20)	44.17	97.73%	95.49%	feasible
Paper	(13.10, 2.10)	44.08	99.99%	89.40%	infeasible

finds a cheaper option: roster (6, 6, 3) with thresholds (12.20, 23.40). Compared with the paper’s fixed (5, 5, 3), it uses 105 more physician-hours per week, but it cuts mean waiting by 33 minutes per patient and reduces the SLA penalty from 4,913 to 60 cost units. The net weekly saving is 28,878 units, or 20.7%.

Optimising roster and thresholds together makes a measurable difference. Roster (6, 6, 3) scores 111,932 when screened at one

Table 10: The base paper’s seven staffing scenarios, priced using Eq. (9). Cost in arbitrary units per week.

	Roster	Phys-h	Wait	SL_4	Total
S_0	(5,5,3)	714	44.18	95.64%	139,391
S_1	(5,5,4)	777	35.19	98.53%	134,121
S_2	(5,5,5)	840	31.25	99.32%	136,490
S_3	(5,6,5)	889	16.34	99.98%	124,942
S_4	(5,7,5)	938	11.45	100.00%	125,327
S_5	(6,7,5)	994	4.32	100.00%	124,082
S_6	(7,7,5)	1050	2.38	100.00%	128,652
Ours	(6,6,3)	819	10.97	99.93%	110,513

Table 11: Sensitivity of the staffing recommendation to the cost weights.

Scenario	α	β	γ	Winner	vs. (5,5,3)
Baseline	120	0.500	50	(6,6,3)	−20.7%
Waiting $\times 4$	120	2.000	50	(6,7,3)	−53.2%
Waiting $\times \frac{1}{4}$	120	0.125	50	(5,6,3)	−5.2%
Physicians $\times 1.5$	180	0.500	50	(6,6,3)	−12.4%
SLA breach $\times 10$	120	0.500	500	(6,6,3)	−39.5%

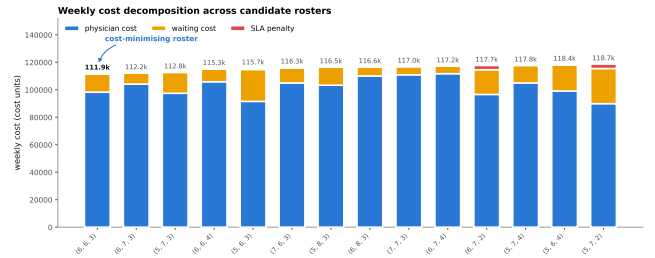


Figure 6: Weekly cost split into physician, waiting and SLA-penalty components for the best screened rosters.

fixed threshold pair and 110,870 after re-tuning to its own optimum (12.20, 23.40). More importantly, the best thresholds change a lot from one roster to another, with k_2 ranging from 11.5 to 25.6 across the six refined candidates. Keeping them fixed while varying staffing would mean scoring each roster with thresholds chosen for a different one.

Table 11 tests how sensitive the recommendation is to the cost weights. The best roster does change with the weights. We see this as a result in itself rather than a weakness of the method: the right staffing level depends on how a hospital values a minute of patient waiting against a physician’s salary, so those weights have to be agreed on before a roster is chosen. In every weighting we tried, though, the best roster is cheaper than the paper’s fixed (5, 5, 3), by between 5.2% and 53.2%.

4.5 Variance reduction

Table 12 shows the effect of CRN on every pairwise comparison. The correlations between paired results range from +0.996 to +0.9996,

Table 12: Variance of the pairwise difference in mean waiting time, with and without common random numbers; 100 replications per arm.

Comparison	Var ind.	Var CRN	Removed	Gain
IFP vs. ALT	241.34	12.44	94.85%	19.4×
IFP vs. SBP	330.48	6.12	98.15%	54.0×
IFP vs. WSEPT	288.20	13.16	95.43%	21.9×
IFP vs. Gcμ	216.19	8.50	96.07%	25.4×
ALT vs. SBP	220.77	2.15	99.03%	102.9×
ALT vs. WSEPT	191.64	0.11	99.94%	1789.3×
ALT vs. Gcμ	223.75	0.46	99.79%	485.4×
SBP vs. WSEPT	198.04	2.23	98.87%	88.7×
SBP vs. Gcμ	268.97	1.03	99.62%	261.8×
WSEPT vs. Gcμ	215.27	0.59	99.72%	362.7×
Median			98.95%	95.8×

What Common Random Numbers buy (target CI half-width 0.5 min)

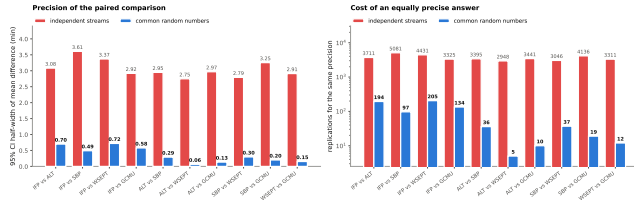


Figure 7: Left: 95% confidence-interval half-width on each pairwise difference. Right: replications needed for ± 0.5 min precision (log scale).

compared with about 0 for independent streams. The median reduction in variance is 98.95%, which corresponds to a median efficiency gain of 95.8×. Confidence intervals on the mean differences become 4 to 40 times narrower.

Two points are worth noting. First, CRN does not bias the results. The per-policy means and standard deviations agree between the two arms within sampling error, as expected, since CRN only changes the joint distribution. Second, it can change conclusions. Several comparisons that do not pass the Bonferroni threshold with independent streams become clearly significant under CRN. Where the two arms disagree, the CRN result is the more reliable one, because it estimates the same quantity with lower variance.

A 95.8× efficiency gain does not mean that one replication is enough. The ratio is itself an estimate, and a t -interval needs enough degrees of freedom. A safer practical reading is that 20–30 CRN replications give comparisons at least as precise as the base paper’s 100 independent ones, which is still a 3–5× saving.

4.6 Robustness

We repeat all three of the base paper’s sensitivity studies and include the added policies. Scaling all arrival rates from -15% to $+15\%$ (Figure 8) moves ρ from 0.739 to 1.000. Under light load the policies perform almost the same, and they separate sharply as the system approaches saturation, with the gap between them growing from 1.6 to 53 min. Across the seven staffing scenarios (Figure 9), waits

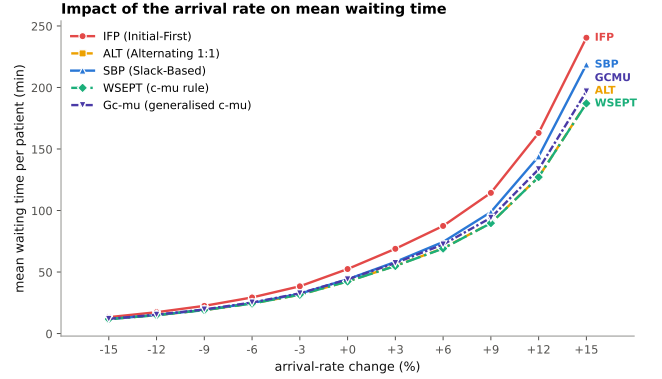


Figure 8: Effect of the arrival rate on mean waiting time, all five policies.

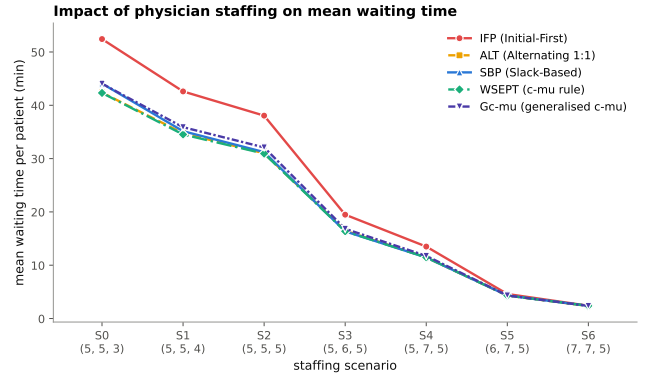


Figure 9: Effect of physician staffing on mean waiting time across the base paper’s seven scenarios.

fall with diminishing returns, and the choice of policy matters most when physicians are scarce: the gap shrinks from 10.1 min at S_0 to 0.06 min at S_6 . Finally, replacing the truncated exponential service times with truncated lognormals that have the same nominal means raises all waits by 44–54% because of the heavier right tail, but the ranking stays exactly the same. The comparative conclusions therefore do not depend on the choice of service-time distribution. All three findings agree with the base paper.

4.7 Threats to validity

Our claims have three main limitations. *Construct validity*: the base paper’s dataset is not available, so we calibrate only to published summary statistics. The 12–25% gap in waiting times is the visible result of this, and only the relative conclusions should be relied on. *Internal validity*: five parameters were chosen by us. Four of them are varied or stress-tested, and for the fifth ($SL^{\min}$) we report both raw service levels so that readers can apply a different threshold themselves. *External validity*: the model covers one ED with one triage system, the $c\mu$ optimality guarantee is weakened by re-entrant flow, and the cost weights are illustrative rather than taken from a real hospital.

5 Conclusion and Future Work

We rebuilt the discrete-event model of Lv et al. [10] using only its published parameters and compared it with every KPI the paper reports. All six structural findings carry over, including the equipment-utilisation table (within 0.5 percentage points) and the conclusions of all three sensitivity analyses. Absolute waiting times are 12–25% higher, which we attribute to the high implied load $\rho \approx 0.87$. We report this gap instead of tuning it away.

We then added four extensions. A policy based on the $c\mu$ rule, which is optimal for a stated delay-cost objective, gives the lowest holding cost (50% below IFP) and the lowest raw mean wait. Our Nelder–Mead implementation with an exterior penalty comes within 0.006 min of the grid-search optimum while using 19× fewer week-long simulations. An explicit cost model shows that roster (6, 6, 3) with re-tuned thresholds is 20.7% cheaper per week than the paper’s fixed (5, 5, 3), although the best roster depends on the cost weights, which we report as a finding. Exact common random numbers remove 98.9% of the variance in every pairwise comparison.

Our most important result is a negative one. On our model, the paper’s published thresholds (13.1, 2.1) break a 95% Level IV service-level constraint, because $k_2 = 2.1$ delays intervention until minute 117.9 of 120. Re-tuning under the constraint fixes this and gives the only tuned policy a hospital could adopt. Scheduling parameters fitted to one system should not be assumed to carry over to another.

We see three directions for future work. The $c\mu$ guarantee could be re-derived for the re-entrant, time-varying setting studied here, for which only asymptotic results currently exist. The optimiser could be applied to larger decision vectors, such as per-shift thresholds or thresholds that depend on the current queue state. There the advantage over grid search grows quickly, and the base paper itself suggests reinforcement learning for this. Finally, the cost model could be set up as a stochastic program, with the roster as a first-stage decision and the thresholds as a recourse decision. This would replace our two-stage screening heuristic with a proper decomposition.

Acknowledgments

We thank the course instructors of CSE 402 for their guidance on scope and methodology. All simulation code, generated result tables, figures and a provenance log recording the exact command behind every output are available at <https://github.com/Ahnaf-Akib-Tanim/Numeric-Project-submission>.

References

- [1] Cagatay Buyukkoc, Pravin Varaiya, and Jean Walrand. 1985. The $c\mu$ Rule Revisited. *Advances in Applied Probability* 17, 1 (1985), 237–238. doi:10.2307/1427064
- [2] David R. Cox and Walter L. Smith. 1961. *Queues*. Methuen, London, UK.
- [3] Andrew J. Davenport, Christophe Gefflot, and J. Christopher Beck. 2001. Slack-Based Techniques for Robust Schedules. *Proceedings of the Sixth European Conference on Planning (ECP-2001)* (2001), 7–18.
- [4] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. 2020. Array Programming with NumPy. *Nature* 585, 7825 (2020), 357–362. doi:10.1038/s41586-020-2649-2
- [5] Junfei Huang, Boaz Carmeli, and Avishai Mandelbaum. 2015. Control of Patient Flow in Emergency Departments, or Multiclass Queues with Deadlines and Feedback. *Operations Research* 63, 4 (2015), 892–908. doi:10.1287/opre.2015.1389
- [6] John D. Hunter. 2007. Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering* 9, 3 (2007), 90–95. doi:10.1109/MCSE.2007.55
- [7] Sujin Kim, Raghu Pasupathy, and Shane G. Henderson. 2015. A Guide to Sample Average Approximation. *Handbook of Simulation Optimization* (2015), 207–243. doi:10.1007/978-1-4939-1384-8_8
- [8] Averill M. Law. 2015. *Simulation Modeling and Analysis* (5th ed.). McGraw-Hill Education, New York, NY, USA.
- [9] Peter A. W. Lewis and Gerald S. Shedler. 1979. Simulation of Nonhomogeneous Poisson Processes by Thinning. *Naval Research Logistics Quarterly* 26, 3 (1979), 403–413. doi:10.1002/nav.3800260304
- [10] Wei Lv, Runzhang Liu, Feiyi Yan, and Yan Wang. 2026. Discrete Event Simulation-Based Analysis and Optimization of Emergency Patient Scheduling Strategies. *Healthcare* 14, 1 (2026), 99. doi:10.3390/healthcare14010099
- [11] Avishai Mandelbaum and Alexander L. Stolyar. 2004. Scheduling Flexible Servers with Convex Delay Costs: Heavy-Traffic Optimality of the Generalized $c\mu$ -Rule. *Operations Research* 52, 6 (2004), 836–855. doi:10.1287/opre.1040.0152
- [12] John A. Nelder and Roger Mead. 1965. A Simplex Method for Function Minimization. *Comput. J.* 7, 4 (1965), 308–313. doi:10.1093/comjnl/7.4.308
- [13] Jorge Nocedal and Stephen J. Wright. 2006. *Numerical Optimization* (2nd ed.). Springer, New York, NY, USA. doi:10.1007/978-0-387-40065-5
- [14] Melissa E. O’Neill. 2014. PCG: A Family of Simple Fast Space-Efficient Statistically Good Algorithms for Random Number Generation. *ACM Trans. Math. Software* (2014). Technical Report HMC-CS-2014-0905, Harvey Mudd College.
- [15] Jan A. Van Mieghem. 1995. Dynamic Scheduling with Convex Delay Costs: The Generalized $c\mu$ Rule. *The Annals of Applied Probability* 5, 3 (1995), 809–833. doi:10.1214/aoap/1177004706
- [16] Lien Vanbrabant, Kris Braekers, Katrien Ramaekers, and Inneke Van Nieuwenhuyse. 2019. Simulation of Emergency Department Operations: A Comprehensive Review of KPIs and Operational Improvements. *Computers & Industrial Engineering* 131 (2019), 356–381. doi:10.1016/j.cie.2019.03.025
- [17] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. 2020. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods* 17, 3 (2020), 261–272. doi:10.1038/s41592-019-0686-2
- [18] Lien Wang and Erik Demeulemeester. 2023. Simulation Optimization in Healthcare Resource Planning: A Literature Review. *IIE Transactions* 55, 10 (2023), 985–1007. doi:10.1080/24725854.2022.2147606